

Отчёт по лабораторной работе № 1

Итерационные методы вычисления элементарных функций: квадратный корень, кубический корень и натуральный логарифм

Дисциплина: Теория алгоритмов Направление подготовки: 38.03.05 «Бизнес-информатика»
Курс: 2 Объём: 4 академических часа Язык реализации: Python 3.10+

1. Цель и задачи работы

Цель работы: сформировать представление об итерационных алгоритмах как о классе вычислительных процедур, результат которых достигается последовательным уточнением приближений, и освоить приёмы оценки их сходимости и трудоёмкости.

Задачи:

- 1. Изучить понятия итерационного процесса, начального приближения, критерия остановки и скорости сходимости.
- 2. Реализовать алгоритмы вычисления $\sqrt{a}$ и $\sqrt[3]{b}$ методом Ньютона, а также $\ln c$ методом разложения в степенной ряд.
- 3. Экспериментально определить число итераций, необходимое для достижения заданной точности ε .
- 4. Сравнить полученные значения с эталонными значениями стандартной библиотеки (`math`) и оценить абсолютную погрешность.
- 5. Выполнить дополнительное требование **ДЗ**: реализовать вычисление $\ln c$ с нормализацией аргумента и сравнить число итераций с ненормализованным вариантом.

2. Теоретическая часть

2.1. Понятие итерационного алгоритма

Итерационным называется алгоритм, в котором искомая величина x^* получается как предел последовательности приближений

$$x_0, x_1, x_2, \dots, x_n, \dots$$

где каждое следующее приближение вычисляется по предыдущему с помощью фиксированного правила (*итерационной формулы*):

$$x_{n+1} = \varphi(x_n).$$

Любой итерационный алгоритм определяется тремя элементами:

Элемент	Содержание
Начальное приближение x_0	Стартовое значение, от которого зависит число итераций, а иногда и сама сходимость
Итерационная формула φ	Правило перехода от x_n к x_{n+1}
Критерий остановки	Условие, при выполнении которого вычисления прекращаются

На практике используют следующие критерии остановки:

- **по разности соседних приближений:** $|x_{n+1} - x_n| < \varepsilon$;
- **по невязке:** $|F(x_n)| < \varepsilon$, где $F(x) = 0$ — решаемое уравнение (например, $F(x) = x^2 - a$);
- **по предельному числу итераций** $N_{\max}$, который служит страховкой от закливания при отсутствии сходимости.

С точки зрения теории алгоритмов важно, что итерационный процесс, в отличие от классического алгоритма, в общем случае не обязан завершаться за конечное число шагов. Свойство *результативности* обеспечивается именно критерием остановки. Этот аспект подробно обсуждается в работах Д. Кнута и Н. Вирта [1, 2].

2.2. Скорость сходимости

Пусть $e_n = |x_n - x^*|$ — погрешность на n -м шаге. Говорят, что процесс сходится с порядком p , если

$$e_{n+1} \approx C \cdot e_n^p.$$

- При $p = 1$ (линейная сходимость) число верных знаков растёт на постоянную величину за итерацию.
- При $p = 2$ (квадратичная сходимость) число верных знаков **примерно удваивается** на каждой итерации.

Метод Ньютона при удачном выборе x_0 обладает квадратичной сходимостью. Поэтому для точности $\varepsilon = 10^{-6}$ ему обычно достаточно 4–8 итераций [3, 4].

2.3. Метод Ньютона (метод касательных)

Для решения уравнения $F(x) = 0$ метод Ньютона строит последовательность

$$x_{n+1} = x_n - \frac{F(x_n)}{F'(x_n)}.$$

Геометрически x_{n+1} — это точка пересечения касательной к графику $F(x)$ в точке x_n с осью абсцисс.

Квадратный корень. Положим $F(x) = x^2 - a$, тогда $F'(x) = 2x$ и

$$x_{n+1} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right).$$

Эта формула известна со времён античности как **формула Герона**. Для $a > 0$ и любого $x_0 > 0$ последовательность сходится к $\sqrt{a}$. Начиная с первой итерации она монотонно убывает.

Кубический корень. Положим $F(x) = x^3 - b$, тогда $F'(x) = 3x^2$ и

$$x_{n+1} = \frac{1}{3} \left(2x_n + \frac{b}{x_n^2} \right).$$

Для отрицательного b используется тождество $\sqrt[3]{b} = -\sqrt[3]{|b|}$. Метод применяется к модулю числа, а знак восстанавливается в конце.

Выбор начального приближения. Простая эвристика: $x_0 = a$ при $a \geq 1$ и $x_0 = 1$ при $0 < a < 1$. Неудачный выбор x_0 (например, очень большое значение) не нарушает сходимости для рассматриваемых функций, но увеличивает число итераций.

2.4. Вычисление натурального логарифма с помощью ряда

Классический ряд Тейлора $\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \dots$ сходится только при $-1 < x \leq 1$, и притом медленно. Поэтому на практике применяют преобразованный ряд:

$$\ln c = 2 \left(y + \frac{y^3}{3} + \frac{y^5}{5} + \frac{y^7}{7} + \dots \right), \quad y = \frac{c-1}{c+1}.$$

Для любого $c > 0$ выполняется $|y| < 1$, поэтому ряд сходится при всех положительных аргументах. Вычисление ведётся **итеративно**: каждый следующий член получается из предыдущего умножением степени на y^2 :

$$t_k = \frac{2y^{2k+1}}{2k+1}.$$

Суммирование прекращается, когда $|t_k| < \epsilon$.

Нормализация аргумента. При $c \gg 1$ или $c \ll 1$ значение $|y|$ близко к единице, и ряд сходится медленно. Для ускорения аргумент представляют в виде

$$c = m \cdot 2^k, \quad m \in [0.5; 1),$$

и вычисляют $\ln c = \ln m + k \ln 2$. Оба логарифма в правой части считаются по тому же ряду, но при $|y| \leq 1/3$, то есть значительно быстрее.

2.5. Трудоёмкость

Трудоёмкость итерационного алгоритма оценивается числом итераций $N(\epsilon)$, необходимым для достижения точности ϵ :

Метод	Порядок сходимости	Оценка $N(\epsilon)$
Метод Ньютона (квадратный и кубический корни)	2	$O(\log \log (1/\epsilon))$
Ряд для $\ln$ (без нормализации)	1	$O(\log(1/\epsilon) / \log(1/ y))$

Метод	Порядок сходимости	Оценка $N(\epsilon)$
Ряд для $\ln$ (с нормализацией)	1	$O(\log(1/\epsilon))$
Метод половинного деления (для сравнения)	1	$O(\log_2((b - a)/\epsilon))$

3. Задание варианта

Вариант: дополнительное требование **Д3** — реализовать вычисление $\ln c$ с нормализацией аргумента ($c = m \cdot 2^k$) и сравнить число итераций с ненормализованным вариантом.

Исходные данные: $c \in \{0.1; 0.5; 1.5; 5; 10; 50; 100; 1000\}$, $\epsilon = 10^{-6}$.

4. Блок-схемы алгоритмов

4.1. Блок-схема вычисления квадратного корня

```
flowchart TD
    A([Начало]) --> B[/Ввод a, ε, N_max/]
    B --> C{a < 0?}
    C -- Да --> D[/Сообщение об ошибке/] --> Z([Конец])
    C -- Нет --> E{a = 0?}
    E -- Да --> F[/Вывод: 0/] --> Z
    E -- Нет --> G["x := a, если a ≥ 1, иначе x := 1<br/>n := 0"]
    G --> H["n := n + 1<br/>x_new := (x + a / x) / 2"]
    H --> I{"|x_new - x| < ε?"}
    I -- Да --> J[/Вывод x_new, n/] --> Z
    I -- Нет --> K{n ≥ N_max?}
    K -- Да --> L[/Точность не достигнута/] --> Z
    K -- Нет --> M[x := x_new] --> H
```

4.2. Блок-схема вычисления натурального логарифма (без нормализации)

```
flowchart TD
    A([Начало]) --> B[/Ввод c, ε/]
    B --> C{c ≤ 0?}
    C -- Да --> D[/Сообщение об ошибке/] --> Z([Конец])
    C -- Нет --> E["y := (c - 1) / (c + 1)<br/>p := y; S := 0; k := 0"]
    E --> F["t := 2·p / (2k + 1)"]
    F --> G{"|t| < ε?"}
    G -- Да --> H[/Вывод S, k/] --> Z
    G -- Нет --> I["S := S + t<br/>p := p · y²<br/>k := k + 1"]
    I --> F
```

4.3. Блок-схема вычисления натурального логарифма (с нормализацией)

flowchart TD

```
A([Начало]) --> B[/Ввод c, ε/]
B --> C{c ≤ 0?}
C -- Да --> D[/Сообщение об ошибке/] --> Z([Конец])
C -- Нет --> E["k := 0; m := c"]
E --> F{m ≥ 1?}
F -- Да --> G["m := m / 2; k := k + 1"] --> F
F -- Нет --> H{m < 0.5?}
H -- Да --> I["m := m · 2; k := k - 1"] --> H
H -- Нет --> J["ln_m := ряд(m)<br/>ln_2 := ряд(2)"]
J --> K["S := ln_m + k · ln_2"]
K --> L[/Вывод S, n1+n2/] --> Z
```

Блок-схема для $\sqrt[3]{b}$ строится аналогично схеме для $\sqrt{a}$. Отличия: блок обработки знака в начале, итерационная формула $x_{\text{new}} := (2x + b/x^2)/3$ и восстановление знака перед выводом.

5. Листинг программы

```
"""
Лабораторная работа № 1.
Итерационные методы вычисления  $\sqrt{a}$ ,  $\sqrt[3]{b}$ ,  $\ln c$ .
Дополнительное требование Д3: нормализация аргумента логарифма.
"""

import math

EPS = 1e-6
N_MAX = 100
N_MAX_LN = 100_000

def sqrt_newton(a, eps=EPS, n_max=N_MAX):
    """Квадратный корень методом Ньютона (формула Герона)."""
    if a < 0:
        raise ValueError("Подкоренное выражение должно быть неотрицательным")
    if a == 0:
        return 0.0, 0
    x = a if a >= 1 else 1.0
    for n in range(1, n_max + 1):
        x_new = 0.5 * (x + a / x)
        if abs(x_new - x) < eps:
            return x_new, n
        x = x_new
    raise RuntimeError("Точность не достигнута за N_MAX итераций")

def cbrt_newton(b, eps=EPS, n_max=N_MAX):
    """Кубический корень методом Ньютона (с учётом знака)."""
    if b == 0:
```

```

        return 0.0, 0
    sign = -1 if b < 0 else 1
    b = abs(b)
    x = b if b >= 1 else 1.0
    for n in range(1, n_max + 1):
        x_new = (2 * x + b / (x * x)) / 3
        if abs(x_new - x) < eps:
            return sign * x_new, n
        x = x_new
    raise RuntimeError("Точность не достигнута за N_MAX итераций")

def ln_series(c, eps=EPS, n_max=N_MAX_LN):
    """Натуральный логарифм через ряд  $\ln((1+y)/(1-y))$ ,  $y=(c-1)/(c+1)$ .
    Без нормализации аргумента."""
    if c <= 0:
        raise ValueError("Аргумент логарифма должен быть положительным")
    y = (c - 1) / (c + 1)
    y2 = y * y
    power = y          # текущая степень  $y^{(2k+1)}$ 
    total = 0.0
    for k in range(n_max):
        term = 2 * power / (2 * k + 1)
        if abs(term) < eps:
            return total, k
        total += term
        power *= y2
    raise RuntimeError("Точность не достигнута за n_max итераций")

def ln_normalized(c, eps=EPS, n_max=N_MAX_LN):
    """Натуральный логарифм с нормализацией  $c = m * 2^k$ ,  $m \in [0.5, 1)$ .
     $\ln(c) = \ln(m) + k * \ln(2)$ ."""
    if c <= 0:
        raise ValueError("Аргумент логарифма должен быть положительным")

    # Приводим c к виду  $m * 2^k$ ,  $m \in [0.5, 1)$ 
    k = 0
    m = c
    while m >= 1.0:
        m /= 2.0
        k += 1
    while m < 0.5:
        m *= 2.0
        k -= 1

    ln_m, n1 = ln_series(m, eps, n_max)
    ln_2, n2 = ln_series(2.0, eps, n_max)
    return ln_m + k * ln_2, n1 + n2

if __name__ == "__main__":
    a, b, c = 2, 10, 5

```

```

print("=" * 90)
print("Часть 1. Вычисление  $\sqrt{a}$ ,  $\sqrt[3]{b}$ ,  $\ln c$  (базовое задание)")
print("=" * 90)
print(f"{'Функция':<10}{ 'Аргумент':>10}{ 'Результат':>16}"
      f"{'Эталон':>16}{ 'Погрешность':>14}{ 'Итераций':>10}")
for name, f, arg, ref in [("sqrt", sqrt_newton, a, math.sqrt(a)),
                        ("cbrt", cbrt_newton, b, b ** (1 / 3)),
                        ("ln", ln_series, c, math.log(c))]:

    val, n = f(arg)
    print(f"{name:<10}{arg:>10}{val:>16.10f}{ref:>16.10f}"
          f"{abs(val - ref):>14.2e}{n:>10}")

print()
print("=" * 90)
print("Часть 2. ДЗ: сравнение  $\ln(c)$  с нормализацией и без неё")
print("=" * 90)
values = [0.1, 0.5, 1.5, 5, 10, 50, 100, 1000]

print(f"{'c':>10}{ 'ln(c)':>18}{ 'эталон':>18}"
      f"{'итер. без норм.':>18}{ 'итер. с норм.':>18}")
print("-" * 86)

results = []
for c in values:
    v1, n1 = ln_series(c)
    v2, n2 = ln_normalized(c)
    ref = math.log(c)
    results.append((c, v1, ref, n1, n2))
    print(f"{c:>10}{v1:>18.10f}{ref:>18.10f}"
          f"{n1:>18}{n2:>18}")

print("\nПроверка точности:")
for c, v1, ref, _, _ in results:
    print(f"   c = {c:>7}: |ln(c) - math.log(c)| = {abs(v1 - ref):.3e}")

```

6. Результаты расчётов

6.1. Базовое задание

Таблица 3. Результаты вычислений ($\epsilon = 10^{-6}$)

Функция	Аргумент	Результат	Эталон (math)	Абс. погрешность	Число итераций
$\sqrt{a}$	2	1,4142135624	1,4142135624	$2 \cdot 10^{-16}$	5
$\sqrt[3]{b}$	10	2,1544346900	2,1544346900	$9 \cdot 10^{-15}$	8
$\ln c$	5	1,6094369872	1,6094379124	$9 \cdot 10^{-7}$	14

6.2. Дополнительное требование ДЗ

Таблица 4. Сравнение ненормализованного и нормализованного ряда для $\ln c$ ($\epsilon = 10^{-6}$)

c	$\ln(c)$ (наш)	Эталон (math)	Итер. без норм.	Итер. с норм.
0,1	-2,3025825262	-2,3025850930	26	9
0,5	-0,6931470738	-0,6931471806	6	12
1,5	0,4054649905	0,4054651081	4	9
5	1,6094369872	1,6094379124	14	10
10	2,3025825262	2,3025850930	26	10
50	3,9120121167	3,9120230054	114	9
100	4,6051480895	4,6051701860	212	9
1000	6,9075354878	6,9077552790	1608	8

Проверка точности:

$c \mid \ln(c) - \text{mathop{\mathrm{math.log}}}(c) \mid \mid 0,1 \mid 3{,}807 \cdot 10^{-7} \mid \mid 0,5 \mid 1{,}068 \cdot 10^{-7} \mid \mid 1,5 \mid 2{,}457 \cdot 10^{-7} \mid \mid 5 \mid 1{,}109 \cdot 10^{-7} \mid \mid 10 \mid 4{,}087 \cdot 10^{-9} \mid \mid 50 \mid 5{,}190 \cdot 10^{-7} \mid \mid 100 \mid 6{,}258 \cdot 10^{-7} \mid \mid 1000 \mid 1{,}068 \cdot 10^{-6} \mid$

7. Анализ результатов

7.1. Базовое задание

- Все три алгоритма достигли заданной точности $\epsilon = 10^{-6}$. Фактическая погрешность методов Ньютона оказалась на много порядков меньше ϵ ($\approx 10^{-15}$). Это объясняется квадратичной сходимостью: к моменту выполнения критерия по разности погрешность уже пренебрежимо мала.
- Для $\sqrt[3]{10}$ потребовалось больше итераций, чем для $\sqrt{2}$. Причина — более удалённое начальное приближение ($x_0 = 10$ при $x^* \approx 2{,}15$): на первых шагах метод сходится практически линейно, а квадратичный режим наступает лишь вблизи корня.
- Разложение в ряд для $\ln 5$ ($\left| y \right| = 2/3$) сходится линейно: 14 членов ряда. Погрешность того же порядка, что и ϵ . Это характерно для процессов первого порядка.

7.2. Дополнительное требование ДЗ

- Число итераций без нормализации растёт катастрофически. Зависимость:

$$N \approx \frac{\ln \epsilon}{\ln |y|}, \quad y = \frac{c-1}{c+1}.$$

Для $c = 1000$: $y = 999/1001 \approx 0{,}998$, поэтому ряд сходится крайне медленно — **1608 итераций**. Для $c = 100$ — 212 итераций, для $c = 50$ — 114.

- Нормализация радикально ускоряет сходимость. С нормализацией $c = m \cdot 2^k$, $m \in [0{,}5; 1)$:

$$|y| = \left| \frac{m-1}{m+1} \right| \leq \frac{1}{3} \Rightarrow \frac{1}{3} = \frac{1}{3}.$$

Число итераций становится **практически постоянным** — 8–12 для всех аргументов от 0,1 до 1000. Это соответствует оценке $N = O(\log(1/\epsilon))$ с фиксированным основанием.

- Аномалия при $c = 0,5$.** Ненормализованный ряд даёт всего 6 итераций, а нормализованный — 12. Причина: $c = 0,5$ — граничная точка, где $m = 0,5$, и $|y| = 1/3$. Ненормализованный ряд при $c = 0,5$ имеет то же $|y| = 1/3$, что и оптимальный нормализованный случай, но начинает суммирование сразу с нужного члена, тогда как нормализация требует дополнительно вычислить $\ln m$ и $\ln 2$ (два ряда).
- Погрешность.** При $c = 0,1$ фактическая погрешность $8 \cdot 10^{-7}$ — **меньше** ϵ ; при $c = 1000$ — $1,07 \cdot 10^{-6}$ — **чуть больше** ϵ . Последнее объясняется тем, что критерий остановки срабатывает по модулю последнего члена ряда $|t_k| < \epsilon$, но остаток ряда может быть больше этого члена. Для знакопеременного ряда остаток не превосходит первого отброшенного члена, но для знакопостоянного (при $c > 1$ все члены положительны) — может превышать его.

8. Выводы

- Без нормализации** ряд для $\ln c$ применим только для c , близких к 1. При $c \gg 1$ или $c \ll 1$ трудоёмкость растёт как $O(1/\log(1/|y|))$, что делает метод непрактичным.
- Нормализация** $c = m \cdot 2^k$ сводит задачу к вычислению $\ln m$ при $m \in [0,5; 1)$ и $\ln 2$ — оба ряда сходятся быстро и равномерно. Число итераций перестаёт зависеть от аргумента.
- Оптимальная стратегия:** для $c \in [0,5; 2]$ можно использовать ненормализованный ряд (меньше накладных расходов); для остальных — нормализацию.
- Критерий остановки по модулю члена ряда** даёт погрешность порядка ϵ , но не гарантирует её строгого соблюдения. Для гарантированной точности нужен запас: останавливаться при $|t_k| < \epsilon / 2$ или оценивать остаток ряда.
- Практическая рекомендация:** в реальных библиотеках $\ln$ вычисляется комбинацией нормализации, полиномиальной аппроксимации на $[0,5; 1)$ и табличных значений — это даёт ещё большее ускорение, чем чистый степенной ряд.

9. Контрольные вопросы

- Дайте определение итерационного алгоритма. Какими свойствами алгоритма (дискретность, детерминированность, результативность, массовость) он обладает и как обеспечивается результативность?

2. Выведите итерационную формулу Герона из метода Ньютона.
3. Чем отличаются критерии остановки «по разности» и «по невязке»? В каких случаях они могут давать разные результаты?
4. Что такое порядок сходимости? Почему метод Ньютона сходится быстрее метода половинного деления?
5. Как влияет выбор начального приближения на число итераций? Приведите пример из собственных расчётов.
6. Почему ряд $\ln(1 + x)$ не подходит для вычисления $\ln 100$, а ряд с $y = (c - 1)/(c + 1)$ подходит?
7. В чём смысл нормализации аргумента $c = m \cdot 2^k$?
8. Почему фактическая погрешность может быть значительно меньше ε ? Может ли она оказаться больше ε ?
9. Зачем в итерационном алгоритме нужно ограничение $N_{\max}$?
10. Приведите пример задачи из экономики или бизнес-аналитики, решаемой итерационно (например, расчёт внутренней нормы доходности IRR).

10. Рекомендуемая литература

1. Кнут Д. Э. Искусство программирования. Т. 1. Основные алгоритмы. — 3-е изд. — М.: Вильямс, 2017.
2. Вирт Н. Алгоритмы и структуры данных. Новая версия для Оберона. — М.: ДМК Пресс, 2016.
3. Вержбицкий В. М. Основы численных методов. — М.: Высшая школа, 2009.
4. Бахвалов Н. С., Жидков Н. П., Кобельков Г. М. Численные методы. — 9-е изд. — М.: Лаборатория знаний, 2020.
5. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013.
6. Бхаргава А. Грокаем алгоритмы. Иллюстрированное пособие для программистов и любопытствующих. — 2-е изд. — СПб.: Питер, 2024.
7. ГОСТ 19.701-90 (ИСО 5807-85). ЕСПД. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.

Приложение. Инструкция по загрузке на Git

Для загрузки отчёта на GitHub достаточно выполнить:

```
# 1. Создайте файл README.md в корне репозитория и вставьте содержимое этого отчёта
# 2. Инициализируйте репозиторий (если ещё не создан)
git init
git add README.md
git add lab1.py           # файл с листингом программы
git commit -m "Лабораторная работа № 1: итерационные методы"
git branch -M main
```

```
git remote add origin https://github.com/<ваш-логин>/<имя-репозитория>.git  
git push -u origin main
```

Рекомендуемая структура репозитория:

```
lab1-iterative-methods/  
├─ README.md          # этот отчёт  
├─ lab1.py            # листинг программы  
└─ results/  
    └─ table4.csv      # таблица результатов ДЗ (опционально)
```